Informe post-incidente — Simulacro de restauración (U6)

Proyecto: Food Store · **Motor:** PostgreSQL 17.11 · **Fecha:** 2026-11-03

Integrantes: Enzo Giaquinta · Agustin Contardi · Fernando Torrez · Ramiro Salcedo · Ignacio Sánchez

1. Respaldo utilizado

Dato	Valor
Archivo	`backups/foodstore_20261103_095841.dump`
Fecha y hora	2026-11-03 09:58:44
Formato / tamaño	Custom (`pg_dump -Fc`) · 6,69 MB
Duración del respaldo	3,68 s

2. RPO y RTO

4,92 s
RPO observado (antigüedad del respaldo)

5,23 s
RTO observado (restauración + validación)

Métrica	Previsto (laboratorio)	Observado
RPO	≤ 24 h (respaldo diario)	4,92 s de antigüedad del respaldo al iniciar el simulacro
RTO	≤ 15 min	5,23 s (creación + `pg_restore` + validaciones)

> El RPO observado mide la antigüedad del respaldo usado, no el intervalo de la política. El RPO de diseño es 24 h; el simulacro se ejecutó 4,92 s después de generar el respaldo, por eso el valor es bajo.

3. Resultado de las validaciones

Tabla	Esperado	Restaurado	OK
`categoria`	10	10	✓
`cliente`	20.000	20.000	✓
`producto`	50.000	50.000	✓
`pedido`	200.000	200.000	✓
`detalle_pedido`	700.140	700.140	✓

- `productos_invalidos = 0` (precio o stock negativos); `detalles_invalidos = 0` (cantidad ≤ 0 o precio unitario < 0).
- Tamaño restaurado: 98 MB (origen: 103 MB). Diferencia física por reorganización del clúster; el contenido lógico es idéntico según los conteos.

4. Problema real encontrado

`dropdb --if-exists` emite un `NOTICE` por stderr cuando la base no existe. Con `$ErrorActionPreference = 'Stop'`, PowerShell 5.1 interpretó ese `NOTICE` como error terminante y abortó el primer simulacro. Se corrigió encapsulando las llamadas nativas en `Invoke-Native`, que baja temporalmente la preferencia y evalúa el código de salida real. Se reejecutó y terminó OK.

También se detectó que la consigna usa `precio` y `precio_unitario`, mientras el esquema real define `precio_lista` y `precio_unitario_facturado`. Las validaciones se adaptaron al esquema real y se dejó constancia; no se inventaron resultados.

5. Mejora concreta

Programar el respaldo diario con el Programador de tareas y alertar si no hay un `FIN OK` en 24 h. Para bases mayores, restaurar con `pg_restore -j 4` (paralelo), agregar retención (7 diarios + 4 semanales) y verificar el dump con `pg_restore --list`. Automatizar el simulacro semanal y publicar RTO/RPO en un tablero.

6. Uso de OpenCode y verificación

La especificación y los scripts se generaron/ajustaron con asistencia de IA. Se verificó la sintaxis con el parser de PowerShell, se ejecutaron realmente el respaldo, el chequeo de salud, el mantenimiento y la restauración, y se compararon los conteos contra la línea base registrada al respaldar. El bug del `NOTICE` se detectó en la primera ejecución real, se corrigió y se reejecutó con éxito. No se incluyó ninguna contraseña.

7. Evidencias

`logs/backup_20261103_095841.log` · `logs/baseline_20261103_095841.txt` · `evidencias/health_check_20261103_095718.txt` · `evidencias/maintenance_20261103_095723.txt` · `logs/restore_test_20261103_095849.log` · `evidencias/restore_test_20261103_095849.txt`